

DATA103/203 Foundational Python (Prof. Maull) / Fall 2026 / HW2

Points Possible	Due Date	Time Commitment (estimated)
20	Thursday, October 15 @ midnight	<i>up to 15 hours</i>

- **GRADING RUBRIC:** Grading will be aligned with the completeness of the objectives.

Your solution is evaluated on:

1. whether your solution correctly answers the question(s) being asked,
2. using the Python language as required in the question(s).

Your solution is **NOT** evaluated on:

1. the elegant use of Python,
2. optimal or high performance Python code unless *explicitly noted* in the question(s).

- **INDEPENDENT WORK:** Copying, cheating, plagiarism and academic dishonesty *are not tolerated* by University or course policy. Please see the syllabus for the full departmental and University statement on the academic code of honor.

- **LARGE LANGUAGE MODEL (LLM) / ARTIFICIAL INTELLIGENCE/LLM (AI) POLICY:**

LLMs / Artificial Intelligence technologies such as Claude, Gemini, Grok, etc are **NOT** to be used in the production of your final homework solutions.

Such technologies offer important capabilities for your *learning* journey, but they are not appropriate in this course for the production of your solutions.

You *may* use such technologies to *learn* more about specific Python concepts or to help clarify your knowledge about the details of Python syntax, modules, techniques or other programming principles.

Final solutions found to be the *partial or entire product* of LLM/AI use **will receive 0 points**, without clarification, resubmission or correction.

OBJECTIVES

- PART 1: Practice writing functions using data from JSON files.
- PART 2: More practice writing functions, explaining what they do and writing DocString documentation for a function.
- BONUS: Do more advanced functions with the input data.

WHAT TO TURN IN

You will enjoy the highest benefits of the starter notebook if you clone the HW Github repository from your Jupyter Hub terminal with the command:

```
git pull https://github.com/kmhuads/s26_data203.git
```

This will ensure you have the most updated files and starter notebook.

Once you have cloned the repository, you can edit the starter notebook with your solution code.

When you are done with your work, it will be best to zip your hw2 folder and all sub-folders with the terminal command (one level outside your notebook folder):

```
zip -r data203_hw2_maull.zip ./hw2
```

This will produce the file with all necessary supporting files (notebooks, data output, etc.) then download it from the Jupyter Hub to your local machine, then upload the .zip to Teams.

If are confused on how to do this, please ask, or visit one of the many tutorials on the basics of using zip in Linux.

If you choose not to use the provided notebook, you will still need to turn in a .ipynb Jupyter Notebook and corresponding files according to the instructions in this homework.

ASSIGNMENT TASKS

(75%) PART 1: Practice writing functions using data from JSON files.

We had a lot of fun in the last HW, now we are going to use data that is a little more serious to exercise our use of functions.

You have, no doubt, seen the in the news this year, alerts about *food recalls* of all kinds. Depending on your home discipline, maybe you have to keep track of and/or spend time understanding these recalls more than the average person has to.

In any event, such recalls can cause disruption in food supplies (depending on their severity, duration and foods impacted), and they certainly could impact *you* directly if there just happen to be recalls on foods that you eat frequently (or infrequently).

To alert the public of “official” recalls the FDA (Food and Drug Administration) maintains a site with all of the food recalls going on and the historical data on prior recalls. Access to this data is even available programmatically.

See these links to education yourself on the official sources of FDA food recall information:

- [FDA Food Recalls Information Webpage](#)
- [FDA Food Recall Resources](#)
- [Food Enforcement API webpage](#)

I have collected a dataset of almost 200 food recalls from 2025 – these represent more than half of the documented food recalls issued by the FDA for that year.

The dataset is in a JSON file in the data/ folder:

- [data/2025_sample_food_recall_dataset.json](#)

In this part of our work we are going to practice writing functions in Python, so here we go.

\$ Task: 1.0 Write a function to return the number of recalls given a category.

When you look at the sample data file, you will notice that the structure of the file is a *list of dictionaries*.

So that the data looks like:

```
[
  {
    # recall #1
  },
  {
    # recall #2
  },
  ...
]
```

Each recall is a dictionary with keys:

- product_description
- category
- status
- city
- and so on ...

You can consider each of these recall dictionaries as *records* with *attributes* – or alternatively objects with attributes that contain keys and values.

Write a function which takes **two** parameters `c` and `ds`, for *category* and *dataset*.

There are 13 categories in our data:

- dairy
- breads, crackers and cereals
- snacks and convenience foods
- frozen foods and ready to eat meals
- fruit juice and speciality beverages
- produce
- seafood
- condiments and spices
- other unclassified items
- supplements and vitamins
- coffee and tea beverages
- non-dairy milk substitute beverages
- eggs

You will use the `category` key of each record and return the number of records which contain that category.

Your function is formally specified below:

FUNCTION SPECIFICATION

function **NAME** : `get_category_count(c, ds)`

function **INPUT** :

- `c` → a category as a string (e.g. "seafood")
- `ds` → a list containing the recall records (as dictionaries)

function **OUTPUT** : the number of records with that category

CODE EXAMPLE:

```
get_category_count("seafood", fda_2025_food_recalls)
```

Example Output:

13

Consult the starter notebook for additional hints.

Use the test functions in the starter notebook to check your work.

\$ Task: 1.1 Write a function `category_summary()` which returns a dictionary of the counts for each category.

You will build on (and reuse) `get_category_count()` from **Task 1.0** to make a new function which calls `get_category_count()` and returns a dictionary of the form:

```
{
  'dairy': 35,
  'breads, crackers and cereals': 30,
  'snacks and convenience foods': 30,
  'frozen foods and ready to eat meals': 26,
  'fruit juice and speciality beverages': 16,
  'produce': 15,
  'seafood': 13,
  'condiments and spices': 11,
  'other unclassified items': 8,
  'supplements and vitamins': 5,
  'coffee and tea beverages': 4,
  'non-dairy milk substitute beverages': 1,
  'eggs': 1
}
```

Where each of the counts represent the number of recalls in that category. You will **only** use the categories in the list given in the starter notebook.

Here are some thoughts to get you started:

1. start with an empty dictionary using the categories in `category_list` as keys,
2. loop through all the records in the input dictionary,
3. use your function from **Task 1.0** to assign the count of the category to the key in your dictionary from **Task 1.0**.

Just remember, your function will return a dictionary.

FUNCTION SPECIFICATION

function NAME: `category_summary(ds)`

function INPUT

- `ds` → a list of dictionaries containing the recall records

function OUTPUT

- the dictionary containing the number of recall records in each category

CODE EXAMPLE:

```
category_summary(fda_2025_food_recalls)
```

Example Output:

```
{
    'dairy': 35,
    'breads, crackers and cereals': 30,
    'snacks and convenience foods': 30,
    'frozen foods and ready to eat meals': 26,
    'fruit juice and speciality beverages': 16,
    'produce': 15,
    'seafood': 13,
    ...
    'condiments and spices': 11,
    'other unclassified items': 8,
    'supplements and vitamins': 5,
    'coffee and tea beverages': 4,
    'non-dairy milk substitute beverages': 1,
    'eggs': 1
}
```

Try not to overthink this. You may also explore (and use) some of the built-ins like `sum()`.

\$ Task: 1.2 Write a function which returns the average recall duration.

For this part, you will need to iterate over all records and use the `recall_initiation_date` and `termination_date` keys to determine the duration.

You will need to use `Python datetime objects` to do so. Study it, you can easily convert the date strings in the date keys and even do time/date math operations very easily.

Please see the guidance in the starter notebook.

FUNCTION SPECIFICATION

function NAME: `get_average_recall_duration(ds)`

function INPUT

- `ds` → a list of dictionaries containing the recall records

function OUTPUT

- a number representing the average recall duration over all records

CODE EXAMPLE:

```
get_average_recall_duration(fda_2025_food_recalls)
```

Example Output:

365

Like before, you can use Python built-ins (e.g. `sum()`, `map()`, etc.) if you find them useful. You **cannot**, however, use other libraries or modules.

\$ Task: 1.3 Write a function which aggregates all items by category and returns the dictionary of recall items.

Your function will effectively reshape the original data such that you build up a dictionary with the keys being the categories in `category_list` and the values of each key being a list of the recalls in the category. Remember, you will use the full dictionary object in your category list.

Here is an example of the output dictionary:

```
{
  'dairy':
  [
    {
      'product_description': ...,
      'category': ...,
      'status': ...,
      ...
    },
    ...
  ]
  'produce':
  [
    {
      'product_description': ...,
      'category': ...,
      'status': ...,
      ...
    },
    ...
  ]
  ...
}
```

Please see the guidance in the starter notebook.

FUNCTION SPECIFICATION

function NAME: `reshape_data(ds)`

function INPUT

- `ds` → a list of dictionaries containing the recall records

function OUTPUT

- a dictionary containing the `category_list` as its keys and the value of each key being the list of recall objects whose category matches

CODE EXAMPLE:

```
reshape_data(fda_2025_food_recalls)
```

Example Output:

You might find some of the code from your **Task 1.1** useful – and you may reuse some of it if you'd like.

(25%) PART 2: More practice writing functions, explaining what they do and writing DocString documentation for a function.

Writing documentation for your functions requires a little practice – and while we are probably ending the era where we are writing our own documentation, you might as well get a tiny bit of practice with it since most AI/LLM tools will likely automate this process out.

We'd still like to know *how* to make this documentation.

In Python, there is a standard for documentation called DocStrings. When you work with future tools and direct them to remember to write documentation in this form, the benefits extend far beyond the source code as such documentation can be used to generate full documentation for a project.

To understand how it is done you will need to study Python DocStrings here:

- [Python DocStrings coding standards by Google](#)

In the notebook are some scaffolding code. You will need to study it and use it in the solutions being asked.

\$ Task: 2.0 Practice explaining code.

Use the code in `mystery_function` in the first cell of the provided notebook and answer the question below:

1. Explain in your own words what function `mystery_function` in the cell does.

Your explanation should include the description of the inputs, and you will need to review the Python `match..case` and the dictionary method `dict.get()` for details of the function.

2. Write DocStrings documentation for the function. Use the style provided by the Google coding standards link already provided. **Include your mystery function explanation in the DocString documentation.**

\$ Task: 2.1 Write a function which uses the output of `mystery_function()`.

Your function will not take any parameters.

It will return a dictionary of the form:

```
{
    'I': 0,
    'II': 0,
    'III': 0
}
```

FUNCTION SPECIFICATION

function **NAME:** `mystery_solved`

function **INPUT:**

- *no input parameters to this function*

function **OUTPUT:**

- a dictionary that contains three keys I, II, III,
- the values of these keys are derived from the `mystery_function` (HINT: you will use those keys populate the dictionary with the values of successive calls to the `mystery_function`).

Code Example:

```
mystery_solved()
```

Example Output:

```
{
  'I': 0,
  'II': 0,
  'III': 0
}
```

Your output WILL NOT contain 0 in the value of the keys, but rather real numbers derived from the call to `mystery_function()`.

Don't overthink this – your solution will be a very straightforward.

(0%) BONUS: Do more advanced functions with the input data.

There are so many things we can now do with our data.

If you want to earn some more points, or just try something more challenging attempt one or more of the options below.

You can earn *up to* several BONUS points *per* task.

\$ Task: B.1 Write a function that returns the list of recalls given a string/substring.

To do this, you will need to look at the `reason_for_recall` key in the recall dictionary record.

You can use a variety of mechanisms to search for the substring.

A robust solution will use the Python Standard Library `regex` object, but points **will** be awarded for solutions which do not use that.

Name your function `find_recalls()` and it will take a two parameters a String parameter `s` and the original recall dictionary `ds` that we have already been using for the bulk of the assignment. The return data will be a List of dictionaries – the recall records that match.

Show that your function works by providing tests for it. For example, this test will return the recall records for `Cesium-137`.

```
find_recalls('Cesium-137', fda_2025_food_recalls)
```

\$ Task: B.2 If you completed B.1 show the code that answers the following questions.

To get points, use your function from **B.1** and show the code which answers the questions. You may also need to use prior functions or write your own for some of the questions:

1. Which months and days were *clostridium botulinum* recalls **initiated**?
2. How many recalls involved undeclared ingredients?
3. Which states had *salmonella* recalls?

\$ Task: B.3 Write a function that returns a dictionary of the count of recalls by state.

Now we will take the `distribution_pattern` key and count up the frequency of recalls by states.

Don't overthink it – there are a few clever, but straightforward and uncomplicated ways to solve this problem.

- You can earn partial points – show as much of your thought process and implementation as possible, even if you do not get the solution to fully work.

Your return dictionary will look like:

```
{
  'AK': 0,
  'LA': 22,
  'AR': 2,
  'AZ': 0,
  'CA': 1,
  'CO': 0,
  'CT': 14,
  ...
}
```

§ Task: B.4 If you completed B.3 show the code that answers the following questions.

1. Which state has the most recalls in 2025?
2. How many states have fewer than 5 recalls in 2025?
3. How many states have more than 18 recalls in 2025?
4. Look at the state with **the most recalls**. Explore the recall data and provide any insights about patterns you see in those recalls (if any).